# Lab 2: Exceptions and Testing

2026-09-11

## Table of contents

Lab Exercise . . . . . 1
Acknowledgements . . . . . 2

Today we will spend some time getting familiar with some of the programming tools that can help make your code more robust and resilient to errors and boundary conditions - tools like unit tests and exceptions (and next week we will spend some time on debugging misbehaving code).

Testing is what you do when you finish implementing a piece of code and want to try it out to see if it works. Running your code manually and seeing if it works is a workable strategy for simple one-time scripts that do simple tasks, but there are situations (like writing a function that others will repeatedly use, or like running the same piece of code on hundreds of files or URLs) where it is prudent to test your code ahead of its actual use or deployment. In today's lab, we are going to use `pytest` to test for correct behavior and verify that exceptions are raised in the right situations.

### Lab Exercise

1- Write a function in Python that finds a number in a string using regular expressions. The number can be positive or negative, integer-valued, or real, and numbers less than one can also be of the form 0.96 or .96. It can be assumed that no numbers appear in scientific notation. Refer to String processing and regular expressions in Python for a quick overview of Python's `re` package. You may use AI to generate the regex, but be sure you understand the output.

2- Write an interface for that function (a function name and arguments), but do not implement the function yet (you can have it return an `None`, or an empty string for now). We will do this in a good old fashioned .py file (not a notebook or a quarto file).

3- Build a test suite using the `pytest` package to test that your function works as intended. Add at least 8 test cases with justification for each. Try to cover the main use cases, and as many potential corner cases or boundary conditions as possible.

4- Now run the test suite. It should fail for all your tests (unless one of them was passing an empty string).

5- Implement the function. You can do this at one go, or case by case. As you implement a case, you can rerun the test suite and see some of the tests relevant to that cases stopping to fail. When all the

tests pass, you are done. This is called test-driven development.

6- If some cases are still failing, that's alright, for the purposes of this lab we don't need to ensure all edge cases are dealt with.

7- Make sure you raise an exception to trap invalid input types.

8- Add a test to check that an exception is properly raised when the function is given invalid input types.

## Acknowledgements

This lab was originally authored by Ahmed Eldeeb and adapted for the Fall 2026 semester.